

```
timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"
```

```
module Maindec_tb;
```

```
    // Testbench Signals
```

```
    reg clk;
```

```
    reg rst_n;
```

```
    reg [6:0] i_opcode;
```

```
    wire o_RegSrcE;
```

```
    wire o_Sel1E;
```

```
    wire o_Sel2E;
```

```
    wire o_LoadE;
```

```
    wire o_BranchE;
```

```
    wire o_JalE;
```

```
    wire o_JalrE;
```

```
    wire o_MemSrcE;
```

```
    wire o_AMOEnE;
```

```
    wire [1:0] o_ResultSrcE;
```

```
    // Instantiate the module under test
```

```
    maindec uut (
```

```
        .clk(clk),
```

```
        .rst_n(rst_n),
```

```
        .i_opcode(i_opcode),
```

```
        .o_RegSrcE(o_RegSrcE),
```

```
        .o_Sel1E(o_Sel1E),
```

```
        .o_Sel2E(o_Sel2E),
```

```
        .o_LoadE(o_LoadE),
```

```
        .o_BranchE(o_BranchE),
```

```
        .o_JalE(o_JalE),
```

```
        .o_JalrE(o_JalrE),
```

```
        .o_MemSrcE(o_MemSrcE),
```

```
        .o_AMOEnE(o_AMOEnE),
```

```
        .o_ResultSrcE(o_ResultSrcE)
```

```
    );
```

```
    // Clock Generation
```

```
    always #5 clk = ~clk;
```

```
    initial begin
```

```
        // Initialize signals
```

```
        clk = 0;
```

```
        rst_n = 0;
```

```
        i_opcode = 7'b00000000;
```

```

// Reset sequence
#10 rst_n = 1;

// Test R-type instruction
i_opcode = `R;
#10;

// Test Load instruction
i_opcode = `LD;
#10;

// Test Store instruction
i_opcode = `S;
#10;

// Test Branch instruction
i_opcode = `B;
#10;

// Test JAL instruction
i_opcode = `J;
#10;

// Test JALR instruction
i_opcode = `JR;
#10;

// Test AMO instruction
i_opcode = `AMO;
#10;

// End simulation
#20 $finish;
end

// Monitor outputs
initial begin
    $monitor("Time=%0t | Opcode=%b | RegSrc=%b | Sel1=%b | Sel2=%b | Load=%b |
Branch=%b | Jal=%b | Jalr=%b | MemSrc=%b | AMOEn=%b | ResultSrc=%b",
            $time, i_opcode, o_RegSrcE, o_Sel1E, o_Sel2E, o_LoadE, o_Branche,
o_JalE, o_JalrE, o_MemSrcE, o_AMOEnE, o_ResultSrcE);
end

endmodule

```